

Reviewer Pack — Minimal Rank of Matroid Representations over XOR-AND Tree Wi...

Entry bb834f7e0dcd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 19:00:24 UTC

Minimal Rank of Matroid Representations over XOR-AND Tree Width

Entry ID: bb834f7e0dcd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 19:00:24 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any k -clique instance, there exists a matroid representation with rank at most n^k .’, ‘The minimal rank of the matroid representation of a k -clique instance is at least as large as the width of its associated XOR-AND tree.’, ‘If an instance I is a k -clique and has an XOR-AND tree with width w , then there exists a matroid M such that the rank of M is at most n^k .’]

Rationale (proposer’s reasoning):

[‘Matroid theory provides a framework for studying combinatorial structures with interesting properties.’, ‘The conjecture bridges the gap between matroids and complexity by exploring their relationship with XOR-AND trees, which are central to understanding communication complexity.’, ‘If true, this would imply that certain matroid structures can be used to design efficient algorithms for solving problems like k -clique.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 06699594cb45b13d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all k -clique instances, the ratio of the minimal rank of the matroid representation to the width of the XOR-AND tree is less than or equal to n^k with a mean of at least 0.8 across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - matroid theory AND XOR-AND tree width - minimizing rank matroids over k-clique instances - XOR-AND tree width AND matroid representations

Top relevant hits considered: - [<http://arxiv.org/abs/1810.06267v2>] Small Space Stream Summary for Matroid Center - [<http://arxiv.org/abs/1510.04532v2>] Internally Perfect Matroids - [<http://arxiv.org/abs/1608.04025v1>] Quasi-matroidal classes of ordered simplicial complexes - [<http://arxiv.org/abs/2504.00267v2>] Representability of Flag Matroids - [<http://arxiv.org/abs/1909.08539v2>] Regular matroids have polynomial extension complexity - [<http://arxiv.org/abs/0804.3263v2>] Lifts of matroid representations over partial fields - [<http://arxiv.org/abs/0804.4584v1>] Feature Unification in TAG Derivation Trees - [<http://arxiv.org/abs/1703.02393v2>] On zeros of the characteristic polynomial of matroids of bounded tree-width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n):
        return [[i, j] for i in range(n) for j in range(i+1, n)]

    def construct_xor_and_tree(edges):
        if not edges:
            return 0
        left_edges = [e for e in edges if e[0] < len(edges) // 2]
        right_edges = [e for e in edges if e[0] >= len(edges) // 2]
        return (construct_xor_and_tree(left_edges), construct_xor_and_tree(right_edges))

    def matroid_rank(n, k):
        return n ** k

    def xor_and_tree_width(tree):
        if isinstance(tree, int):
```

```

        return tree
    else:
        return max(xor_and_tree_width(tree[0]), xor_and_tree_width(tree[1])) + 1

n = random.randint(5, 40)
clique = generate_k_clique(n)
xor_and_tree = construct_xor_and_tree(clique)
matroid_rank_value = matroid_rank(n, len(clique))
xor_and_tree_width_value = xor_and_tree_width(xor_and_tree)

if xor_and_tree_width_value == 0:
    return {
        "metric_name": "rank_to_width_ratio",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "XOR-AND tree width is 0"
    }

ratio = matroid_rank_value / xor_and_tree_width_value

return {
    "metric_name": "rank_to_width_ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": ratio <= n ** len(clique),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = f"rank_to_width_ratio={result['metric_value']}, expected<=n^{len(clique)}"
                print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seed}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4bcead5.py", line 70, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4bcead5.py", line 42, in run_trial
    xor_and_tree = construct_xor_and_tree(clique)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4bcead5.py", line 29, in construct_xor_and_tree
    return (construct_xor_and_tree(left_edges), construct_xor_and_tree(right_edges))
           ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4bcead5.py", line 29, in construct_xor_and_tree
    return (construct_xor_and_tree(left_edges), construct_xor_and_tree(right_edges))
           ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4bcead5.py", line 29, in construct_xor_and_tree
    return (construct_xor_and_tree(left_edges), construct_xor_and_tree(right_edges))
           ~~~~~
```

[Previous line repeated 995 more times]

RecursionError: maximum recursion depth exceeded

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the conjecture could not be evaluated according to the pre-registered support condition. | next: Investigate and fix the recursion error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11788
2	propose	ollama_remote	glm4:latest	0	0	10216
3	preregistration	ollama_remote	glm4:latest	0	0	5598
4	novelty	ollama_remote	glm4:latest	0	0	4520
5	novelty	ollama_remote	glm4:latest	0	0	6170
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42975
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15389
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8995
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8625
10	judge	ollama_remote	glm4:latest	0	0	10408

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 124683 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/bb834f7e0dcd.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bb834f7e0dcd.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bb834f7e0dcd.tar.gz (if generated)